

Machine Learning and Model Deployment Project

Three day project using vital signs and patient information

Goal. Use the first 24 hours of ICU data to predict whether a patient will die during the hospital stay. Combine vital signs measured over time with information recorded at admission. This exercise uses adult data to assess skills useful for NOA. It is not a model for medical use with newborns.

Using AI tools. Vibe coding and AI coding assistants are allowed. You must understand everything you submit, including code written by AI. Explain how it works, why you made each choice, and what its limits are. Say which parts used AI. Be ready to fix a problem or make a small change during the review. Work you cannot explain or test will count as incomplete.

Cloud options. Choose AWS, Google Cloud or Azure. You may run the project on your own computer and show how you would set it up in the cloud. You do not need a paid account. See page 2 for the required Terraform files and checks.

Public dataset

PhysioNet Challenge 2012. Use Set A and its outcomes file. Match them using RecordID. Predict In-hospital_death. The records are adults who stayed in ICU for at least 48 hours; explain how this limits the results. A value of -1 means missing data. Never use fields from the outcomes file as model inputs.

Dataset page <https://physionet.org/content/challenge-2012/1.0.0/>

Tasks to complete

Task	What you must do
Prepare data	Save source file versions and checksums (file fingerprints). Check fields, units, values and times. Make vital-sign and admission tables linked by RecordID. Keep track of missing values. Use measurements from 00:00 to 24:00 after admission, including 24:00. Exclude later data.
Build model inputs	Use HR (heart rate), RespRate (breathing rate), Temp (temperature), Age, Gender and ICUType. For each vital sign, calculate its count, average and last value. Add flags for missing data. Convert category fields into numbers and combine the inputs. Lab data and complex methods are optional.
Train and compare	Compare logistic regression with one tree-based model. Split RecordIDs into 70% training, 15% validation and 15% test, with similar death rates and a fixed random seed. Learn data preparation from training only. Choose model settings and the risk cutoff on validation only. Report test PR-AUC, AUROC, sensitivity, specificity and Brier score. Check results when vitals are missing.
Run predictions	Save data/code versions, split IDs, random seed, settings, scores and model files. Run FastAPI POST /predict in Docker with the saved data preparation steps. Return the predicted probability, model version and a data-quality flag. Check input fields and explain what happens for wrong or missing values.
Plan cloud setup	Show where files and Docker images are stored, where the API runs, who has access and where logs go. Options: AWS S3/ECR/ECS/CloudWatch; Google Cloud Storage/Artifact Registry/Cloud Run/Logging; Azure Blob/ACR/Container Apps/Monitor. Submit Terraform checks as described on page 2.
Test and monitor	Test that later data is excluded, record IDs never overlap across groups, and valid/invalid API calls behave correctly. Add one CI workflow (automatic tests after code changes). Log requests, errors and response times. Add a check for too much missing vital data. Explain how to detect changing data, retrain, release a model and return to an earlier version. Do not automate retraining.

Three-day plan (Proposed)

Day	What to finish
1	Load and check data, define input fields, apply the 24-hour limit, split records and build model inputs.
2	Train both models, make choices using validation data, report test scores and check missing-data results.
3	Run the API, tests and monitoring check; check Terraform and finish the README. Cloud demo optional.

Required results

The data preparation, model tests and API must run on your computer. Page 2 lists what to submit and demonstrate. Explain the steps needed for real-world use in the README. Running the system in a cloud account is optional.

What to submit and show

Time allowed. Complete only the project assigned to you. Spend no more than 24 hours over three working days. Tell us how much time you used and what is unfinished. Complete the required tasks first. Extra work is optional.

Cloud setup. Run the main project on your own computer. Put Terraform files for your chosen cloud in /infra. Terraform describes the cloud resources to create. Specify the provider versions. Show the output of terraform init -backend=false and terraform validate. If you have cloud access, also show a terraform plan with private details removed. If not, explain the planned setup and cost. These checks do not prove that the setup works in the cloud.

If you have no cloud access. Run the API or database on your computer, using Docker where useful. A free tool that imitates cloud services is optional. You must still submit Terraform files for a real cloud and explain where you would store data, run code or queries, control access and collect logs. Lack of cloud credits will not lower your score. State which cloud steps you could not test.

Explain in your README. Who needs access and why? How will you protect passwords and the Terraform state file, which records created resources? How long will you keep data? How will you control costs, remove resources and recover from failure? You do not need to build automatic scaling, full cloud monitoring or a managed scheduler. Do not pay for services or upgrade an account for this exercise.

Show us. Use your README commands to rebuild model inputs from the same source files. Recreate the same data groups and similar model scores; state any expected difference. Start the Docker container. Send a valid POST /predict request, an invalid request and one with missing vital signs. Show which data and model version produced one prediction. Run the tests and monitoring check. Make one small change during the review.

Files to submit. Include a README, exact library versions, data loading and preparation code, saved record IDs for each data group, results for both models, saved model and data preparation steps, API examples, tests, one automatic test workflow and /infra. Add a short model summary covering its purpose, results and limits. The README can also explain the system design, input fields, run commands, AI use and plans for real-world use.

Model scores. PR-AUC and AUROC measure how well the model ranks risk. Sensitivity measures how many deaths it identifies; specificity measures how many survivors it identifies. Brier score checks the predicted probabilities against actual outcomes. Lower Brier is better. Explain why you chose this risk cutoff.

Optional extras. Patient-group comparisons, more complex models, cloud deployment and extra monitoring are optional. No minimum model score is required.